

Lab Project:
**Linux Automation & System
Monitoring with Bash**

By: Hadiya Zuberi

1. Lab Task Objectives / Description:

The objective of this lab is to write and execute a suite of Bash shell scripts on Ubuntu Linux inside a VMware virtual machine environment. The scripts automate system monitoring, security auditing, and administrative tasks relevant to SOC and blue team operations including login checking, CPU load alerting, log file analysis, file permission auditing, large file detection, and live system dashboarding.

2. Software's / Tools Required:

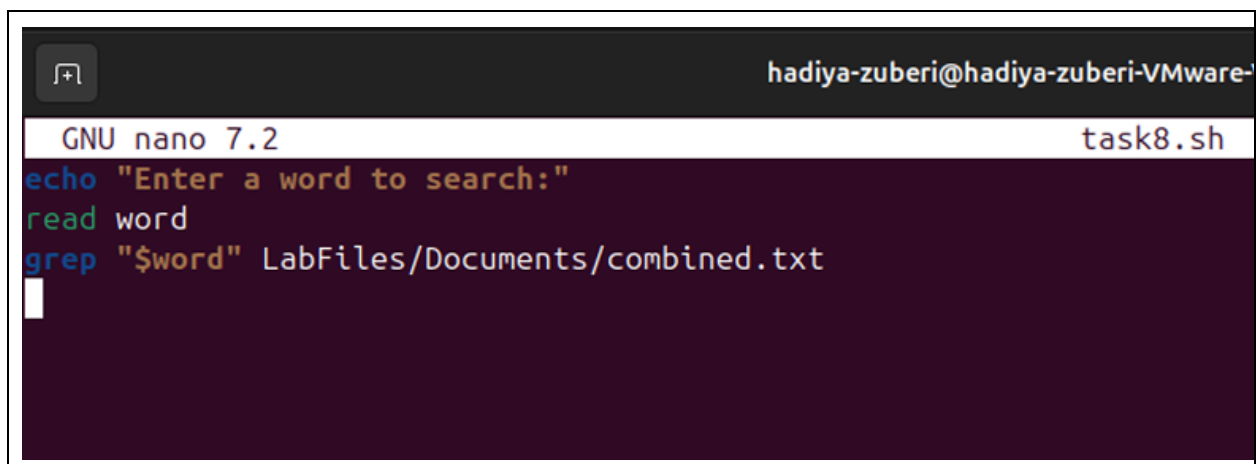
- Hardware: PC, Internet access
- Software: Bash, GNU nano, VMware Workstation
- OS: Ubuntu 24.04 LTS (work machine)

3. Lab Procedure & Results:

Task 01:

Create a script that asks the user to input a word and searches for it in combined.txt. Display all the lines that contain the word.

CODE:

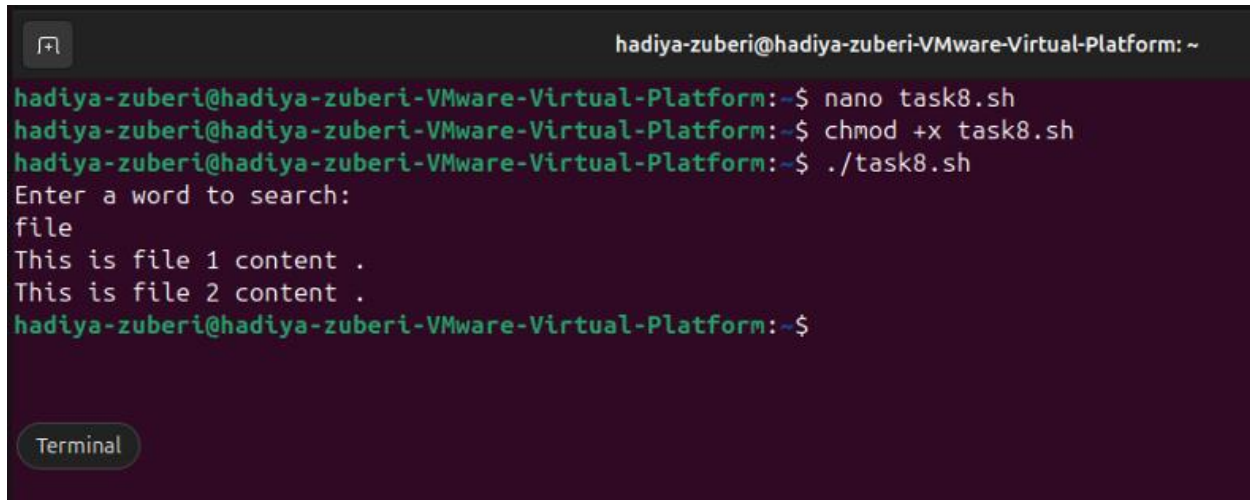


```
hadiya-zuberi@hadiya-zuberi-VMware-  
GNU nano 7.2 task8.sh  
echo "Enter a word to search:"  
read word  
grep "$word" LabFiles/Documents/combined.txt
```

Figure 1.1

OUTPUT:

Use chmod +x to give execution permission



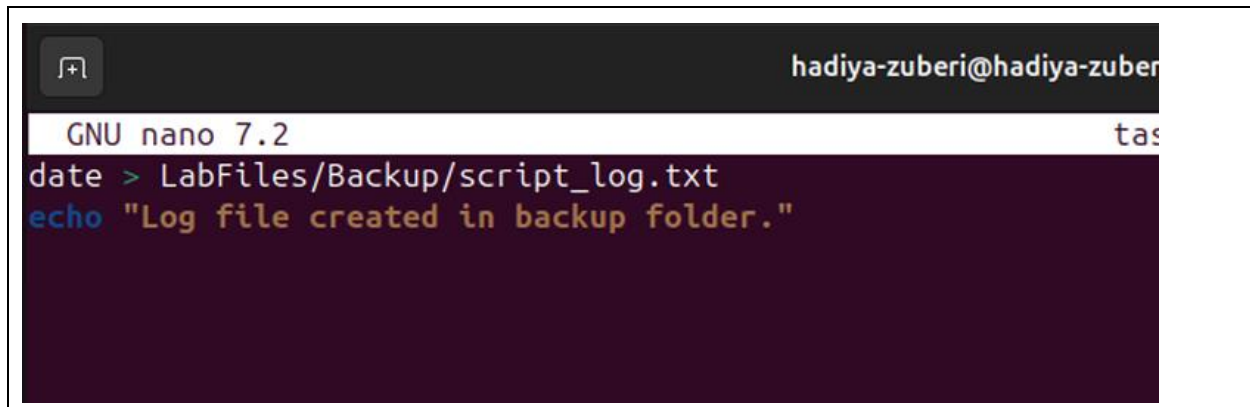
```
hadiya-zuberi@hadiya-zuberi-VMware-Virtual-Platform: ~  
hadiya-zuberi@hadiya-zuberi-VMware-Virtual-Platform:~$ nano task8.sh  
hadiya-zuberi@hadiya-zuberi-VMware-Virtual-Platform:~$ chmod +x task8.sh  
hadiya-zuberi@hadiya-zuberi-VMware-Virtual-Platform:~$ ./task8.sh  
Enter a word to search:  
file  
This is file 1 content .  
This is file 2 content .  
hadiya-zuberi@hadiya-zuberi-VMware-Virtual-Platform:~$
```

Figure 1.2

Task 02:

Write a script that creates a log file named script_log.txt inside Backup folder. The log file should contain the current date and time when the script was executed.

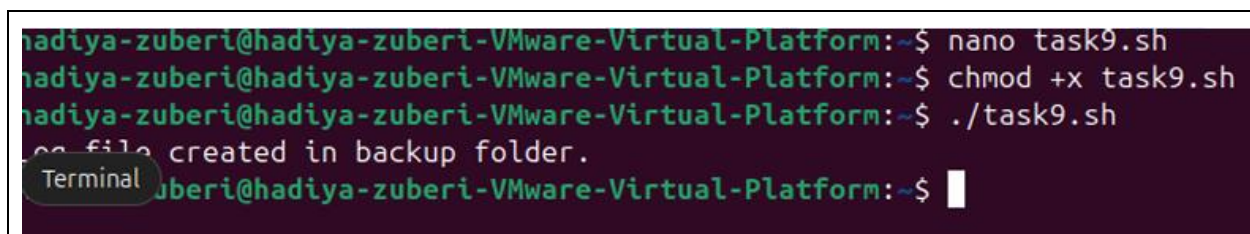
CODE:



```
GNU nano 7.2 tas  
date > LabFiles/Backup/script_log.txt  
echo "Log file created in backup folder."
```

Figure 2.1

OUTPUT:



```
hadiya-zuberi@hadiya-zuberi-VMware-Virtual-Platform:~$ nano task9.sh  
hadiya-zuberi@hadiya-zuberi-VMware-Virtual-Platform:~$ chmod +x task9.sh  
hadiya-zuberi@hadiya-zuberi-VMware-Virtual-Platform:~$ ./task9.sh  
Log file created in backup folder.  
hadiya-zuberi@hadiya-zuberi-VMware-Virtual-Platform:~$
```

Figure 2.2

Write a script to archive the Documents folder into a zip file named Documents_backup.zip inside the Backup folder.

- Sudo apt install zip*

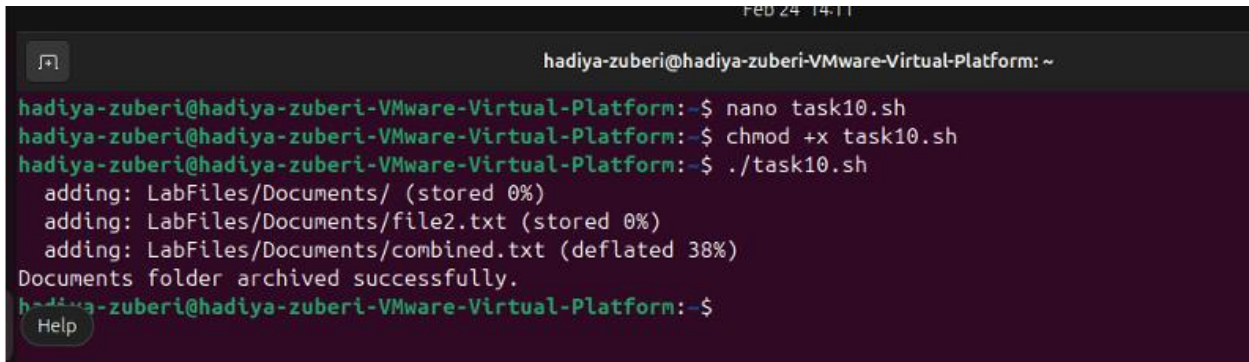
[illegible]

Figure 3.1

```
hadiya-zuberi@hadiya-zuberi-VMware-Virtual-Platform:~$ nano task10.sh
GNU nano 7.2 task10.sh
zip -r LabFiles/Backup/Documents_backup.zip LabFiles/Documents
echo "Documents folder archived successfully."
```

Figure 3.2

OUTPUT:

A terminal window with a dark background. The prompt is 'hadiya-zuberi@hadiya-zuberi-VMware-Virtual-Platform: ~'. The user enters 'nano task10.sh', then 'chmod +x task10.sh', and finally './task10.sh'. The script output shows: 'adding: LabFiles/Documents/ (stored 0%)', 'adding: LabFiles/Documents/file2.txt (stored 0%)', 'adding: LabFiles/Documents/combined.txt (deflated 38%)', and 'Documents folder archived successfully.'. The prompt returns to '~\$'.

```
hadiya-zuberi@hadiya-zuberi-VMware-Virtual-Platform:~$ nano task10.sh
hadiya-zuberi@hadiya-zuberi-VMware-Virtual-Platform:~$ chmod +x task10.sh
hadiya-zuberi@hadiya-zuberi-VMware-Virtual-Platform:~$ ./task10.sh
  adding: LabFiles/Documents/ (stored 0%)
  adding: LabFiles/Documents/file2.txt (stored 0%)
  adding: LabFiles/Documents/combined.txt (deflated 38%)
Documents folder archived successfully.
hadiya-zuberi@hadiya-zuberi-VMware-Virtual-Platform:~$
```

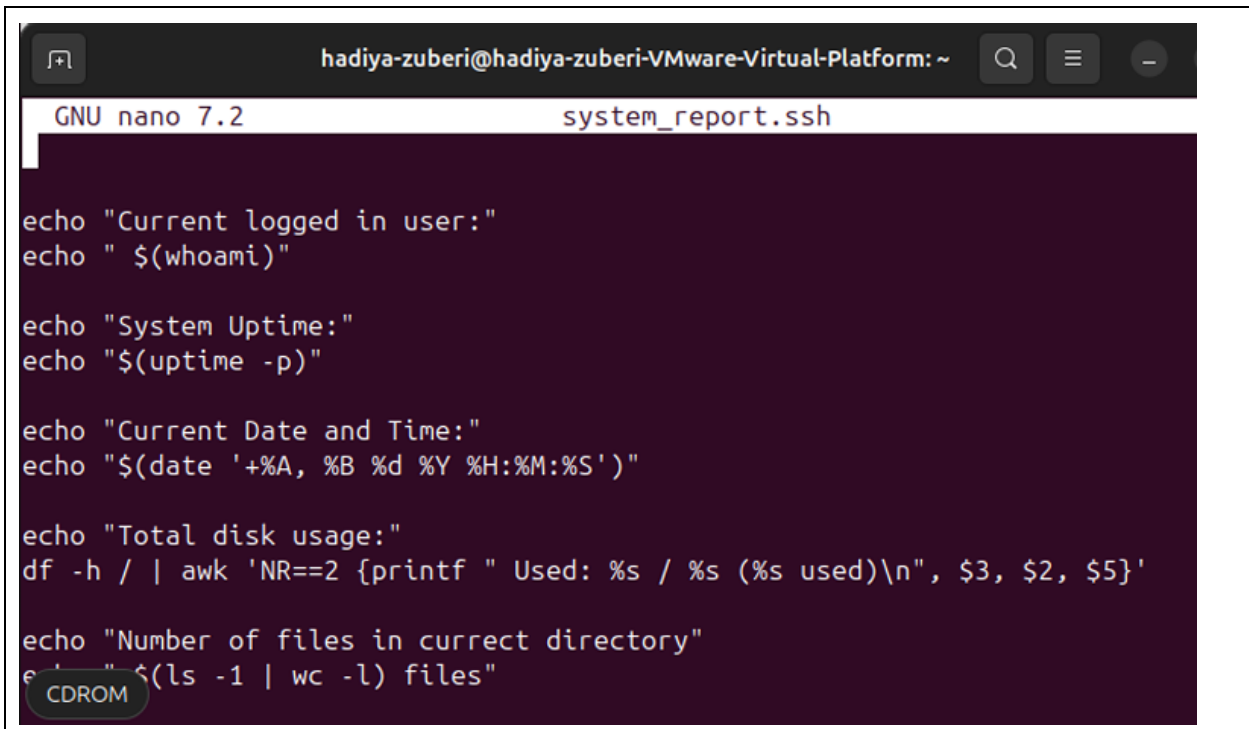
Figure 3.3

Task 04:

Write a script `system_report.sh` that displays:

- Current logged-in user
- System uptime
- Current date and time
- Total disk usage
- Number of files in the current directory

CODE:

A terminal window showing the GNU nano 7.2 editor editing 'system_report.sh'. The script content is: 'echo "Current logged in user:"', 'echo " \$(whoami)"', 'echo "System Uptime:"', 'echo "\$(uptime -p)"', 'echo "Current Date and Time:"', 'echo "\$(date '+%A, %B %d %Y %H:%M:%S')"'', 'echo "Total disk usage:"', 'df -h / | awk 'NR==2 {printf " Used: %s / %s (%s used)\n", \$3, \$2, \$5}''', 'echo "Number of files in current directory"', 'echo "\$(ls -1 | wc -l) files"'. The prompt is 'CDROM'.

```
GNU nano 7.2 system_report.sh

echo "Current logged in user:"
echo " $(whoami)"

echo "System Uptime:"
echo "$(uptime -p)"

echo "Current Date and Time:"
echo "$(date '+%A, %B %d %Y %H:%M:%S')"'

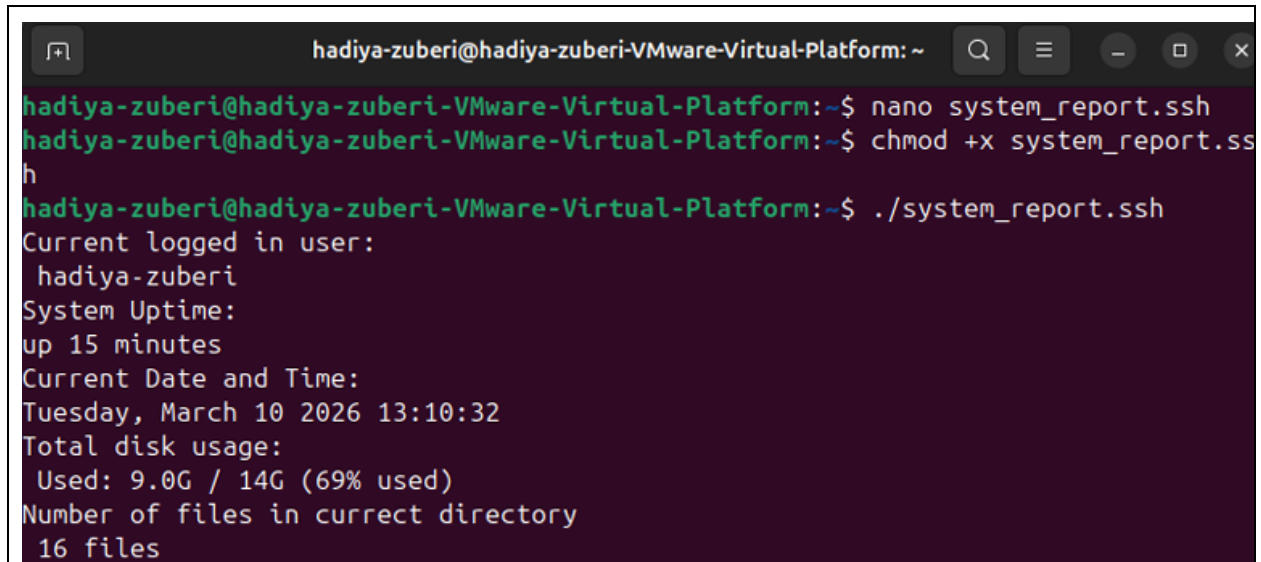
echo "Total disk usage:"
df -h / | awk 'NR==2 {printf " Used: %s / %s (%s used)\n", $3, $2, $5}''

echo "Number of files in current directory"
echo "$(ls -1 | wc -l) files"

CDROM
```

Figure 4.1

OUTPUT:



```
hadiya-zuberi@hadiya-zuberi-VMware-Virtual-Platform: ~  
hadiya-zuberi@hadiya-zuberi-VMware-Virtual-Platform:~$ nano system_report.ssh  
hadiya-zuberi@hadiya-zuberi-VMware-Virtual-Platform:~$ chmod +x system_report.ssh  
hadiya-zuberi@hadiya-zuberi-VMware-Virtual-Platform:~$ ./system_report.ssh  
Current logged in user:  
hadiya-zuberi  
System Uptime:  
up 15 minutes  
Current Date and Time:  
Tuesday, March 10 2026 13:10:32  
Total disk usage:  
Used: 9.0G / 14G (69% used)  
Number of files in current directory  
16 files
```

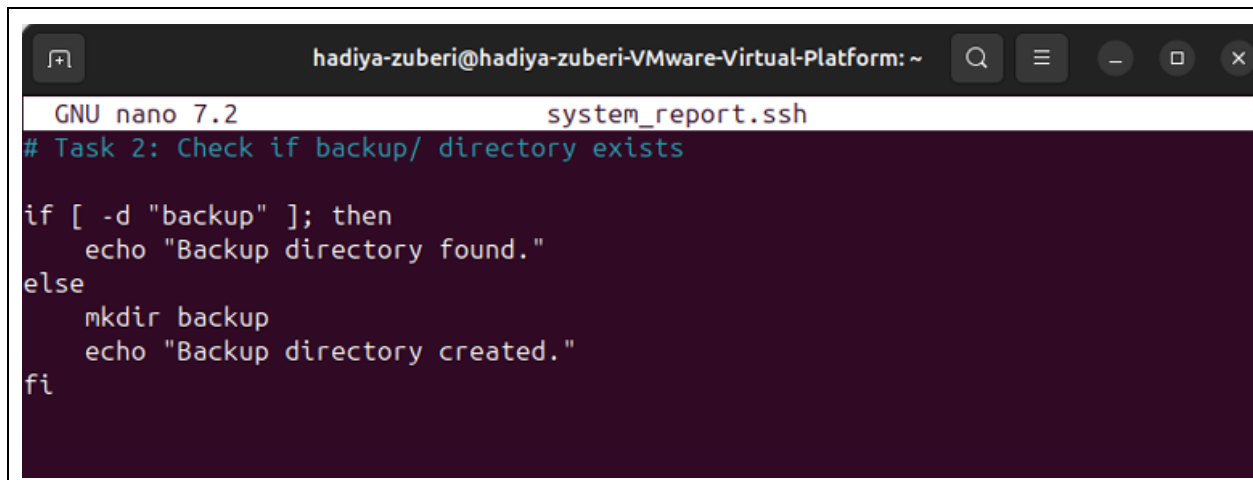
Figure 4.2

Task 05:

Write a script that:

1. Checks if a directory backup/ exists.
2. If it exists → display "Backup directory found".
3. If not → create the directory and display "Backup directory created".

CODE:



```
GNU nano 7.2 system_report.ssh  
# Task 2: Check if backup/ directory exists  
  
if [ -d "backup" ]; then  
    echo "Backup directory found."  
else  
    mkdir backup  
    echo "Backup directory created."  
fi
```

Figure 5.1

OUTPUT:

```
hadiya-zuberi@hadiya-zuberi-VMware-Virtual-Platform:~$ nano system_report.ssh
hadiya-zuberi@hadiya-zuberi-VMware-Virtual-Platform:~$ ./system_report.ssh
Backup directory created.
hadiya-zuberi@hadiya-zuberi-VMware-Virtual-Platform:~$
```

Figure 5.2

Task 06:

Write a script that checks if a file has:

- Read permission
- Write permission
- Execute permission

CODE:

```
GNU nano 7.2 system_report.ssh *
echo -n "Enter filename to check: "
read filename
echo "=====
echo "    PERMISSION CHECK: $filename"
echo "=====

if [ ! -e "$filename" ]; then
    echo " File does not exist."
    exit 1
fi

# Read permission
if [ -r "$filename" ]; then
    echo "Read    permission: YES"
else
    echo "Read    permission: NO"
fi

# Write permission
if [ -w "$filename" ]; then
    echo " Write   permission: YES"
else
    echo " Write   permission: NO"
fi

# Execute permission
if [ -x "$filename" ]; then
    echo " Execute permission: YES"
```



```

else
    echo " Write   permission:  NO"
fi

# Execute permission
if [ -x "$filename" ]; then
    echo " Execute permission:  YES"
else
    echo " Execute permission:  NO"
fi

echo "Symbolic permissions: $(ls -la "$filename" | awk '{print $1}')"

```

Figure 6.1

OUTPUT:

```

hadiya-zuberi@hadiya-zuberi-VMware-Virtual-Platform:~$ ./system_report.ssh
filename to check: task1.sh
=====
PERMISSION CHECK: task1.sh
=====
Read    permission: YES
Write   permission: YES
Execute permission: YES
Symbolic permissions: -rwxrwxr-x
hadiya-zuberi@hadiya-zuberi-VMware-Virtual-Platform:~$

```

Figure 6.2

Task 07:

Create a script that checks CPU load.

Conditions:

Load < 50% → Normal

Load 50–80 → Warning

Load > 80 → Critical

CODE:


```
hadiya-zuberi@hadiya-zuberi-VMware-Virtual-Platform: ~
GNU nano 7.2 system_report.ssh *
echo "======"
echo "      CPU LOAD MONITOR      "
echo "======"

# Get CPU idle % from top, then calculate usage
cpu_idle=$(top -bn1 | grep "Cpu(s)" | awk '{print $8}' | tr -d '%,')

# Fallback using /proc/stat if top format differs
if [ -z "$cpu_idle" ]; then
    cpu_idle=$(top -bn1 | grep "Cpu(s)" | sed 's/.*, *\[([0-9.]*\)\%* id.*\/1/')
fi

# Calculate CPU usage
cpu_load=$((echo "100 - $cpu_idle" | bc 2>/dev/null))

# If bc not available or parsing failed, use alternative
if [ -z "$cpu_load" ] || [ "$cpu_load" = "-" ]; then
    cpu_load=$(grep 'cpu ' /proc/stat | awk '{usage=($2+$4)*100/($2+$4+$5)} END {printf "%.0f", usage}')
fi

echo "Current CPU Load: ${cpu_load}%"
echo ""

# Convert to integer for comparison
cpu_int=$((printf "%.0f" $cpu_load 2>/dev/null || echo $cpu_load))

if [ "$cpu_int" -lt 50 ]; then
    echo "Status: NORMAL (Load is below 50%)"
elif [ "$cpu_int" -le 80 ]; then
    echo "Status: WARNING (Load is between 50% and 80%)"
else
    echo "Status: CRITICAL (Load exceeds 80%)"
fi
```

Figure 7.1

OUTPUT:

```
hadiya-zuberi@hadiya-zuberi-VMware-Virtual-Platform:~$ ./system_report.ssh
=====  
      CPU LOAD MONITOR        
=====
Current CPU Load: 67.0%

Status: WARNING (Load is between 50% and 80%)
hadiya-zuberi@hadiya-zuberi-VMware-Virtual-Platform:~$
```

Figure 7.2

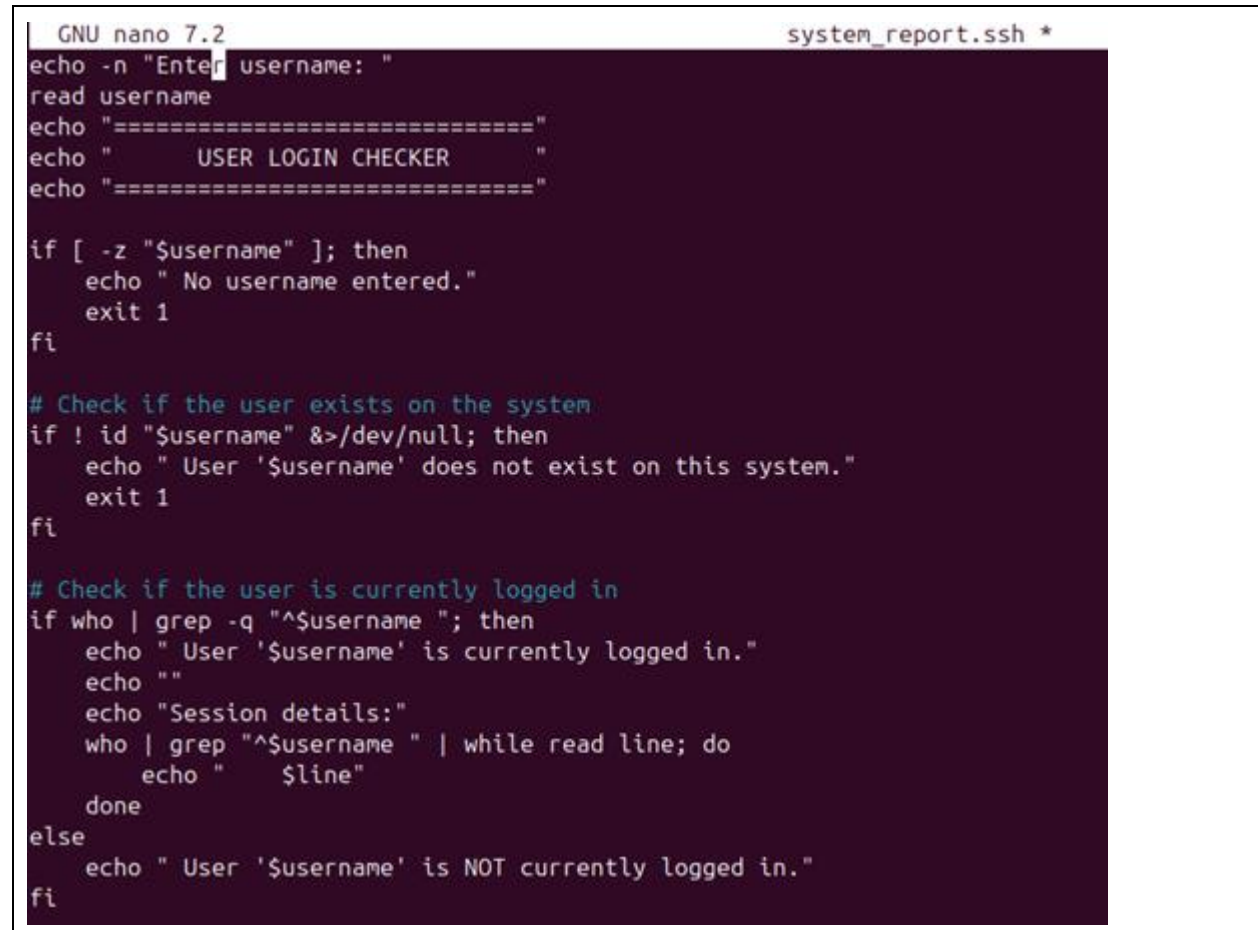
Task 08:

Write a script that checks if a specific user is currently logged in.

Example input:

Enter username: kali

CODE:



```
GNU nano 7.2 system_report.ssh *
echo -n "Enter username: "
read username
echo "=====
echo "      USER LOGIN CHECKER      "
echo "=====

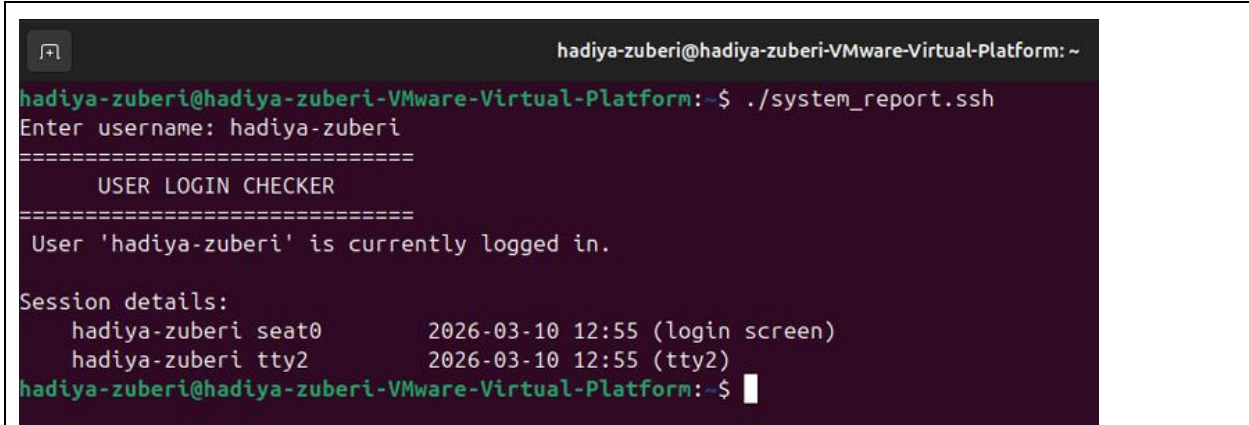
if [ -z "$username" ]; then
    echo " No username entered."
    exit 1
fi

# Check if the user exists on the system
if ! id "$username" &>/dev/null; then
    echo " User '$username' does not exist on this system."
    exit 1
fi

# Check if the user is currently logged in
if who | grep -q "^$username "; then
    echo " User '$username' is currently logged in."
    echo ""
    echo "Session details:"
    who | grep "^$username " | while read line; do
        echo "    $line"
    done
else
    echo " User '$username' is NOT currently logged in."
fi
```

Figure 8.1

OUTPUT:



```
hadiya-zuberi@hadiya-zuberi-VMware-Virtual-Platform: ~  
hadiya-zuberi@hadiya-zuberi-VMware-Virtual-Platform:~$ ./system_report.ssh  
Enter username: hadiya-zuberi  
=====  
USER LOGIN CHECKER  
=====  
User 'hadiya-zuberi' is currently logged in.  
  
Session details:  
hadiya-zuberi seat0      2026-03-10 12:55 (login screen)  
hadiya-zuberi tty2       2026-03-10 12:55 (tty2)  
hadiya-zuberi@hadiya-zuberi-VMware-Virtual-Platform:~$
```

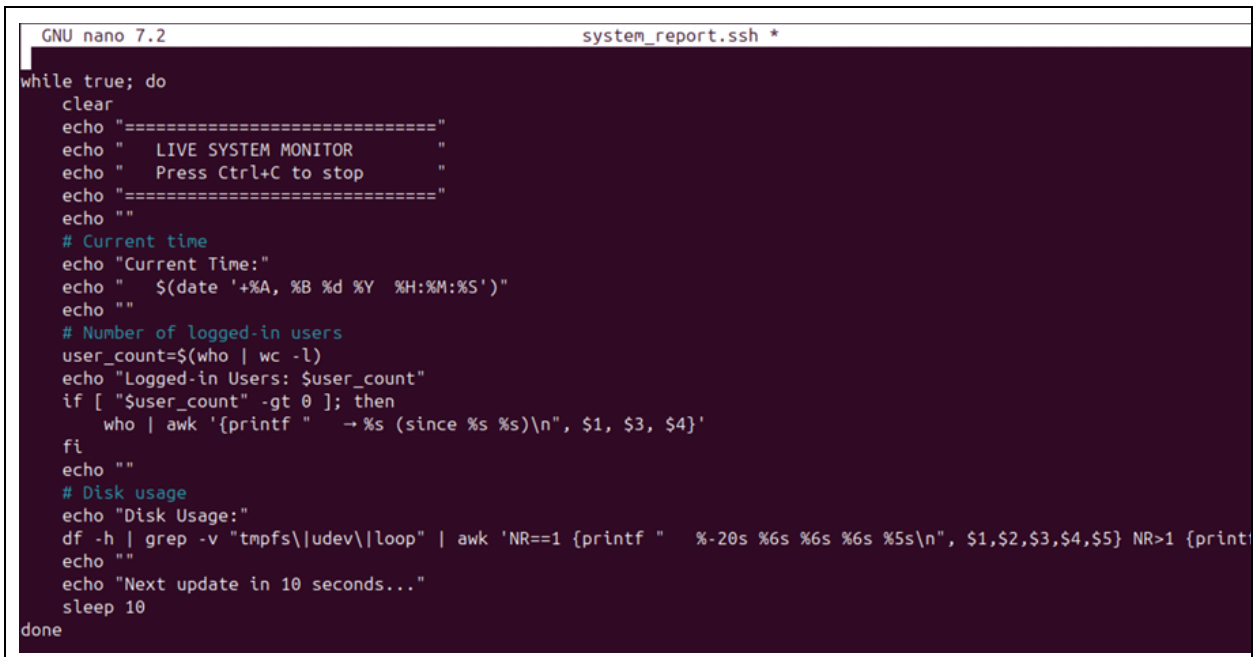
Figure 8.2

Task 09:

Create a script that runs continuously using a while loop and displays every 10 seconds:

- Current time
- Number of logged-in users
- Disk usage

CODE:



```
GNU nano 7.2 system_report.ssh *  
  
while true; do  
    clear  
    echo "===== "  
    echo "    LIVE SYSTEM MONITOR "  
    echo "    Press Ctrl+C to stop  "  
    echo "===== "  
    echo ""  
    # Current time  
    echo "Current Time:"  
    echo "    $(date '+%A, %B %d %Y %H:%M:%S')"  
    echo ""  
    # Number of logged-in users  
    user_count=$(who | wc -l)  
    echo "Logged-in Users: $user_count"  
    if [ "$user_count" -gt 0 ]; then  
        who | awk '{printf "    → %s (since %s %s)\n", $1, $3, $4}'  
    fi  
    echo ""  
    # Disk usage  
    echo "Disk Usage:"  
    df -h | grep -v "tmpfs|udev|loop" | awk 'NR==1 {printf "    %-20s %6s %6s %6s %5s\n", $1,$2,$3,$4,$5} NR>1 {print  
    echo ""  
    echo "Next update in 10 seconds..."  
    sleep 10  
done
```

Figure 9.1

OUTPUT:

```
=====
LIVE SYSTEM MONITOR
Press Ctrl+C to stop
=====

Current Time:
  Tuesday, March 10 2026  14:39:06

Logged-in Users: 2
  → hadiya-zuberi (since 2026-03-10 12:55)
  → hadiya-zuberi (since 2026-03-10 12:55)

Disk Usage:
  Filesystem      Size  Used Avail Use%
  /dev/sda4       14G   9.1G  4.1G  69%
  /dev/sr0        88M   88M    0 100%
  /dev/sr1       5.8G   5.8G    0 100%

Next update in 10 seconds...
```

Figure 9.2

Task 10:

Write a script that checks all files in current directory. If file size is greater than 1MB, display:

Large file detected: filename

Use for loop + if condition.

CODE:

```
GNU nano 1.2 system_report.ssh
echo "=====
echo "      LARGE FILE DETECTOR      "
echo "=====
echo "Scanning: $(pwd)"
echo ""

found=0

for file in *; do
    if [ -f "$file" ]; then
        # Get file size in bytes
        size=$(stat -c%s "$file" 2>/dev/null)

        if [ -n "$size" ] && [ "$size" -gt 1048576 ]; then
            # Convert to human-readable MB
            size_mb=$(echo "scale=2; $size / 1048576" | bc)
            echo " Large file detected: $file (${size_mb} MB)"
            found=$((found + 1))
        fi
    fi
done

echo ""
if [ $found -eq 0 ]; then
    echo " No large files (>1MB) found in current directory."
else
    echo "Total large files found: $found"
fi
```

Figure 10.1

OUTPUT:

```
hadiya-zuberi@hadiya-zuberi-VMware-Virtual-Platform:~$ ./system_report.ssh
=====
      LARGE FILE DETECTOR
=====
Scanning: /home/hadiya-zuberi

No large files (>1MB) found in current directory.
hadiya-zuberi@hadiya-zuberi-VMware-Virtual-Platform:~$
```

Figure 10.2

Normal vs Anomalous Indicators

Feature	Normal System State	Anomalous / Alert State
CPU Load	Below 50% — Normal	Above 80% — Critical alert triggered
Login Status	Expected users only	Unknown user detected as logged in
Log Errors	Few or no error entries	High error count in log file
File Sizes	No oversized files	Files > 1MB flagged for review
Backup Directory	backup/ exists	backup/ missing — auto-created
File Permissions	Expected read/write	Unexpected execute permission found

Conclusion:

In this lab, we designed and executed a suite of Bash shell scripts on Ubuntu Linux inside a VMware virtual machine. Each script was written from scratch using nano and made executable with chmod before running. The scripts covered key security-relevant tasks: system reporting, login session monitoring, CPU load alerting, log file error analysis, file permission auditing, large file detection, and live continuous monitoring using a while-true polling loop.

The exercise reinforced practical Linux administration skills directly applicable to SOC and blue team work. The login checker, log analyzer, and live monitor in particular demonstrate the core logic behind user behaviour analytics and endpoint monitoring agents. All scripts produced correct results under live Ubuntu conditions, validating the logic of conditional statements, grep-based pattern matching, awk field extraction, and threshold-based alerting.